

**Arun Manglick**

Building with AI Agents · LangChain · LangGraph · LangSmith

## AI Travel CoPilot (*A graph-native, multi-agent supervisor architecture under real mutation pressure*)

The intent is not to build another AI assistant that adapts to your preferences, responds to changing travel requirements, and then calls it a "trip plan."

Though I call it **AI Travel CoPilot**, travel is merely the proving ground. The end goal is to use this as a proof of concept for a **multi-agent supervisor architecture** that can seamlessly orchestrate a real **"tax calculation workflow"**, handling complex dependencies while ensuring accuracy, traceability, and controlled execution.

The real objective was to architect a **multi-agent supervisor system** and push a **stateful graph orchestration model** through nine consecutive user modifications, where every agent knows exactly when to wake up, when to remain idle, and when to seek human approval before proceeding.

This effort culminates in a **state-driven orchestration model** capable of coordinating specialist agents through conditional graph edges, selectively re-executing only invalidated subgraphs, checkpointing the entire conversation state, and enforcing structured human-in-the-loop approval gates, all within a single observable execution trace.

After building agents with **CrewAI**, **Oracle AI Studio**, **Copilot**, **n8n**, and several other platforms, I wanted to evaluate the Lang ecosystem as a **vertically integrated agentic architecture**. **LangChain** provides the agent primitives (tool nodes, output parsers, and ReAct loops), **LangGraph** delivers the **graph-native orchestration layer** with typed state, conditional routing, subgraph composition, interrupt gates, and persistent checkpointing, while **LangSmith** provides **real-time observability and evaluation** across every agent handoff, tool invocation, and state transition.

The result is a **state-driven, fully traceable multi-agent system** where the graph itself becomes the orchestrator.

---

### // AGENTIC ORCHESTRATION PATTERNS

Supervisor Architecture

Stateful Graph Orchestration

Sub-Graph Composition

ReAct Agent Loops

Conditional Edge Routing

Typed State Reducers

Human-in-the-Loop Gates

Graph Checkpointing

Selective Re-Execution

Parallel Agent Fan-Out

Error Boundaries

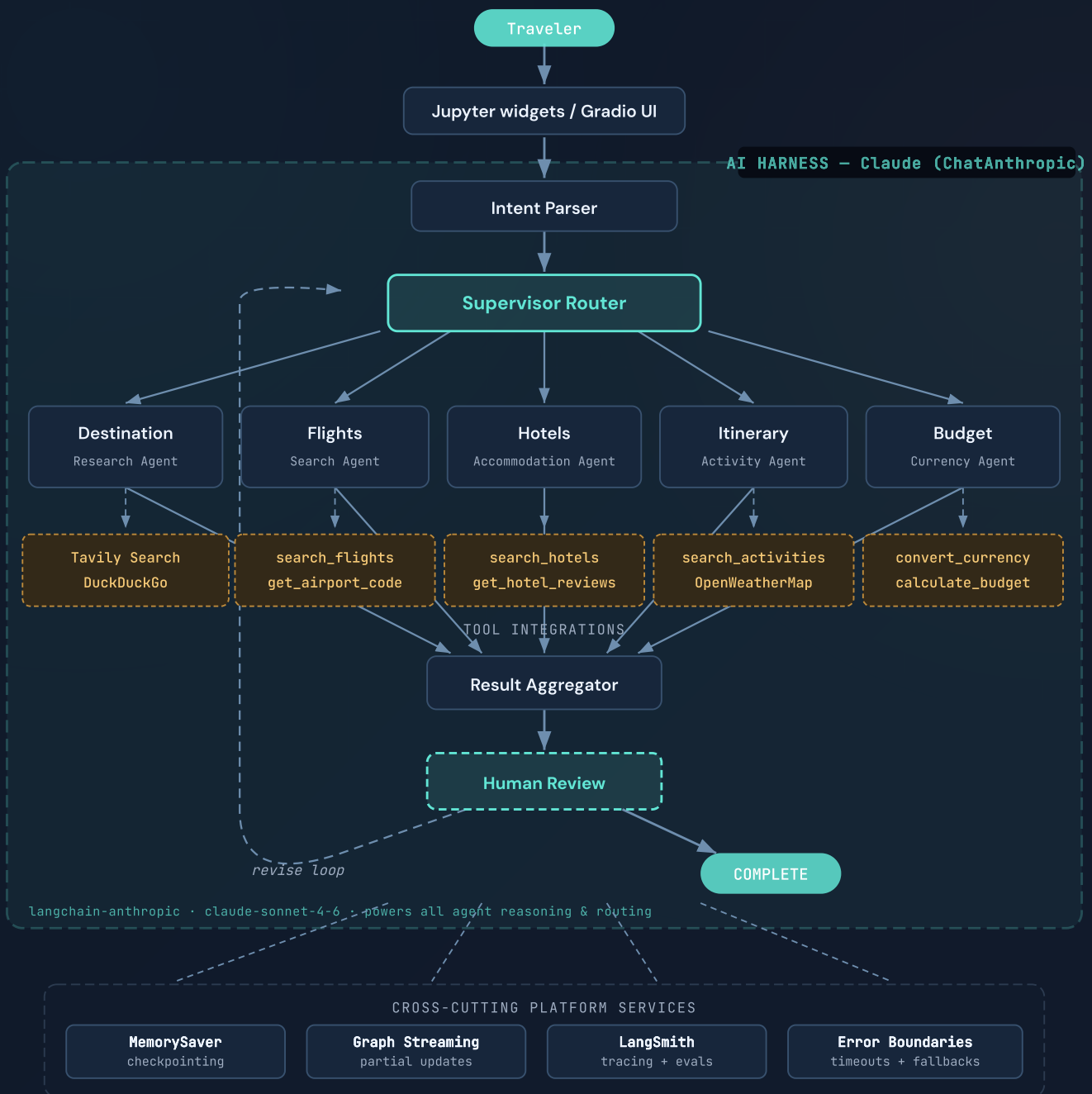
Real-Time Trace Observability

@traceable Evaluation

---

## // THE MULTI-AGENT GRAPH ARCHITECTURE

At the core sits a **Supervisor Router**—the central orchestration node in a LangGraph StateGraph. It doesn't generate text. It reads the typed `TravelPlanState`, evaluates which state slices are `None` or invalidated, and dispatches work to five specialist agents through **conditional edges**—a **state-driven routing table**, not prompt engineering:



LangGraph StateGraph – Supervisor routes to specialists via conditional edges, with tool integrations, human-in-the-loop gate, and cross-cutting platform services

Each specialist is its own **compiled sub-graph**—a self-contained **ReAct agent** with its own execution cycle (reason → tool call → evaluate → reason again) and a dedicated ToolNode. The supervisor never broadcasts. It inspects the shared TravelPlanState through **conditional edge evaluation** and activates only the agents whose state slices are None or invalidated by user mutation. This is **graph-native selective re-execution**—the orchestration topology itself decides what runs.

## // WHY THE LANG FAMILY – A VERTICALLY INTEGRATED AGENTIC STACK

There are many agentic frameworks. I deliberately chose **LangChain + LangGraph + LangSmith** because they form a **vertically integrated agentic stack**—primitives, orchestration, and observability in a single, coherent architecture:

- > **Graph-native state management** — LangGraph's StateGraph with typed TypedDict state, add\_messages reducers, and conditional edges models agent orchestration as a **directed execution graph**, not a chain of function calls. State is explicit, inspectable, and checkpointed—the graph topology *is* the orchestration logic.
- > **Selective re-execution under mutation** — When a user says "find cheaper hotels," the supervisor evaluates TravelPlanState and re-invokes *only* the Hotel agent—not the entire pipeline. This is **native to the graph's conditional edge routing**. In a linear chain framework, you'd rebuild this invalidation logic from scratch.
- > **Sub-graph composition & agent isolation** — Each specialist compiles into its own graph with its own ReAct execution cycle, then plugs into the parent graph as a composable node. Each sub-graph is testable in isolation, independently deployable, and orchestrated by the supervisor—**true multi-agent modularity**.
- > **Structural human-in-the-loop gates** — LangGraph's interrupt\_before pauses graph execution at the human review node, serializes the full plan state, waits for approval or revision input, and resumes the graph exactly where it stopped. Not a confirmation dialog—a **stateful approval gate with revision routing**.
- > **Real-time observability & evaluation** — Every agent handoff, tool invocation, and state transition is auto-traced in LangSmith with zero additional instrumentation. @traceable decorators and custom metadata tags enable per-agent latency profiling, tool-call chain inspection, and **evaluation against curated datasets**—production-grade observability from the first run.

**The architectural stress test:** a single session can trigger 9+ modification scenarios (change dates, swap destination, add a traveler, adjust budget, request dietary options...) and the supervisor must route each mutation to the correct subset of agents without re-executing stable subgraphs. LangGraph's **state-driven conditional edges** turn this into a routing table evaluated at every graph step—not spaghetti code patching around a linear pipeline.

## // THE AGENTIC TECH STACK

Every layer was chosen to serve the architecture. No defaults, no "just use OpenAI."

| LAYER             | TOOL                        | WHY THIS ONE                                                                                                                                            |
|-------------------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| AI Harness        | Claude (ChatAnthropic)      | Powers all agent reasoning & routing — large context window, strong instruction-following for multi-agent dispatch, native tool-use protocol            |
| Orchestration     | LangGraph<br>StateGraph     | The graph-native orchestration layer — conditional edge routing, sub-graph composition, interrupt gates, stateful checkpointing, selective re-execution |
| Agent Primitives  | LangChain                   | ToolNode, ReAct agent pattern, output parsers, prompt templates — the building blocks each sub-graph agent is assembled from                            |
| Web Intelligence  | Tavily + DuckDuckGo         | Tavily: purpose-built search API for AI agents and RAG pipelines. DuckDuckGo: privacy-first fallback with zero API key overhead                         |
| Observability     | LangSmith                   | Real-time trace observability — @traceable decorators, datasets, evaluations, per-agent latency profiling, and tool-call chain inspection               |
| State Persistence | MemorySaver                 | LangGraph's graph checkpointer — thread-based state persistence with full conversation replay, zero external infra                                      |
| Weather Intel     | OpenWeatherMap API          | Context-aware activity routing — the Activity agent uses forecast data to decide outdoor vs. indoor itinerary recommendations                           |
| Currency          | ExchangeRate API            | Real-time conversion for accurate multi-currency budget computation across the Budget agent's state slice                                               |
| Agent UI          | Gradio + Jupyter ipywidgets | Interactive agent interface with streaming output — users observe the multi-agent execution unfold in real time                                         |

// ARCHITECTURAL DIFFERENTIATORS

1. The supervisor is a state-driven router, not a narrator.

It generates zero user-facing text. It reads the typed `TravelPlanState`, evaluates which state slices are `None` or invalidated by user mutation, and dispatches to exactly the right specialist agent through **conditional edge evaluation**. The routing logic lives in the graph topology—not in prompt engineering.

## 2. ReAct execution cycles at every agent node.

Each specialist doesn't call a tool once and return. It runs a full **ReAct loop** within its compiled sub-graph: reason → select tool → invoke → evaluate result → decide whether to invoke again or report. The Destination agent might search, find conflicting visa info, refine the query, cross-reference, and return a structured finding. This is a **self-contained agentic sub-graph with its own execution cycle**, not a single function call.

## 3. Human-in-the-loop is a structural graph gate, not a UI decoration.

After the aggregator assembles a complete plan, the graph *pauses* at a `interrupt_before` node. The full plan state is serialized—flights, hotels, itinerary, budget—and presented for human review. Approval resumes the graph to `COMPLETE`. Revision feedback re-enters the supervisor, which evaluates the mutation and **re-routes to only the affected agent subgraphs**. This is a **stateful approval gate with selective re-execution routing**.

## 4. Graph checkpointing enables selective re-planning under mutation.

MemorySaver checkpoints the full StateGraph per `thread_id`. When a user says "make it Thailand instead," the supervisor reads the checkpointed state, invalidates `destination_research` and every downstream dependency, and **re-executes only the invalidated subgraphs**. Stable agent outputs remain cached. This is **state-driven selective re-execution**—native to the graph, not bolted on.

## 5. Real-time observability and evaluation via LangSmith.

Every agent handoff, every tool invocation, every state transition is auto-traced with zero additional instrumentation. Custom metadata tags (`agent_name`, `tool_name`, `phase`) enable per-agent latency profiling, tool-call chain inspection, and **evaluation against curated travel planning datasets**. The entire multi-agent execution trace is inspectable step by step—from graph entry through supervisor dispatch through tool resolution to human review gate.

### // THE GRAPH UNDER MUTATION PRESSURE

Here's a single multi-turn session—9 consecutive user mutations, each testing the supervisor's **selective re-execution routing**:

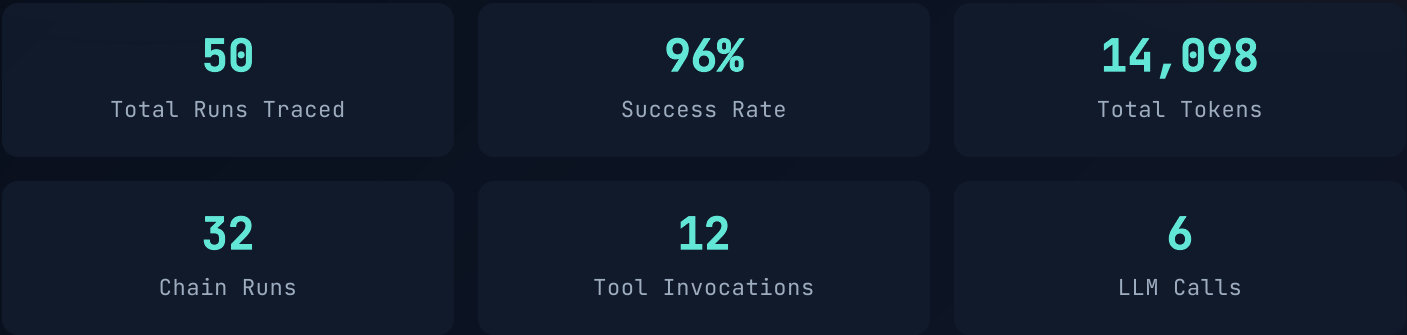
- > "Plan a 7-day trip to the Maldives for 2, budget \$5000" → *full pipeline runs*
- > "Find cheaper hotels" → *only Hotel + Budget agents re-run*
- > "Move it to April 10–17" → *Flight + Hotel + Activity + Budget re-run*
- > "Make it 5 days instead of 7" → *Hotel + Activity + Budget re-run*
- > "One more traveler joining" → *Flight + Hotel + Budget re-run*
- > "Actually, change to Bali" → *all 5 agents re-run, full re-plan*

- > "Add scuba diving" → *only Activity + Budget re-run*
- > "Find a nonstop flight" → *only Flight + Budget re-run*
- > "One of us is vegetarian" → *only Activity + Budget re-run*

Nine mutations. Each one triggers the *minimum subgraph re-execution*—the supervisor evaluates state, invalidates only the affected slices, and dispatches to the correct agent subset. The **graph state persists across every turn** via MemorySaver checkpointing, and the **human approval gate fires after every re-plan**. This is the supervisor pattern proving its architectural value under continuous mutation pressure.

// REAL-TIME OBSERVABILITY & EVALUATION – LANGSMITH TRACE DATA

This isn't hypothetical. Here's what LangSmith's **real-time trace observability** captured from a live multi-agent planning session on ArunManglickLanggraph/AITravelGenie—every agent handoff, tool invocation, and state transition, fully inspectable:



| AGENT & CHAIN TRACES      |   |
|---------------------------|---|
| should_continue (routing) | 7 |
| ChatAnthropic (LLM)       | 6 |
| agent (ReAct loop)        | 6 |
| tools (ToolNode)          | 4 |
| supervisor_router         | 3 |
| human_review (gate)       | 2 |
| LangGraph (graph entry)   | 2 |
| aggregate_results         | 1 |

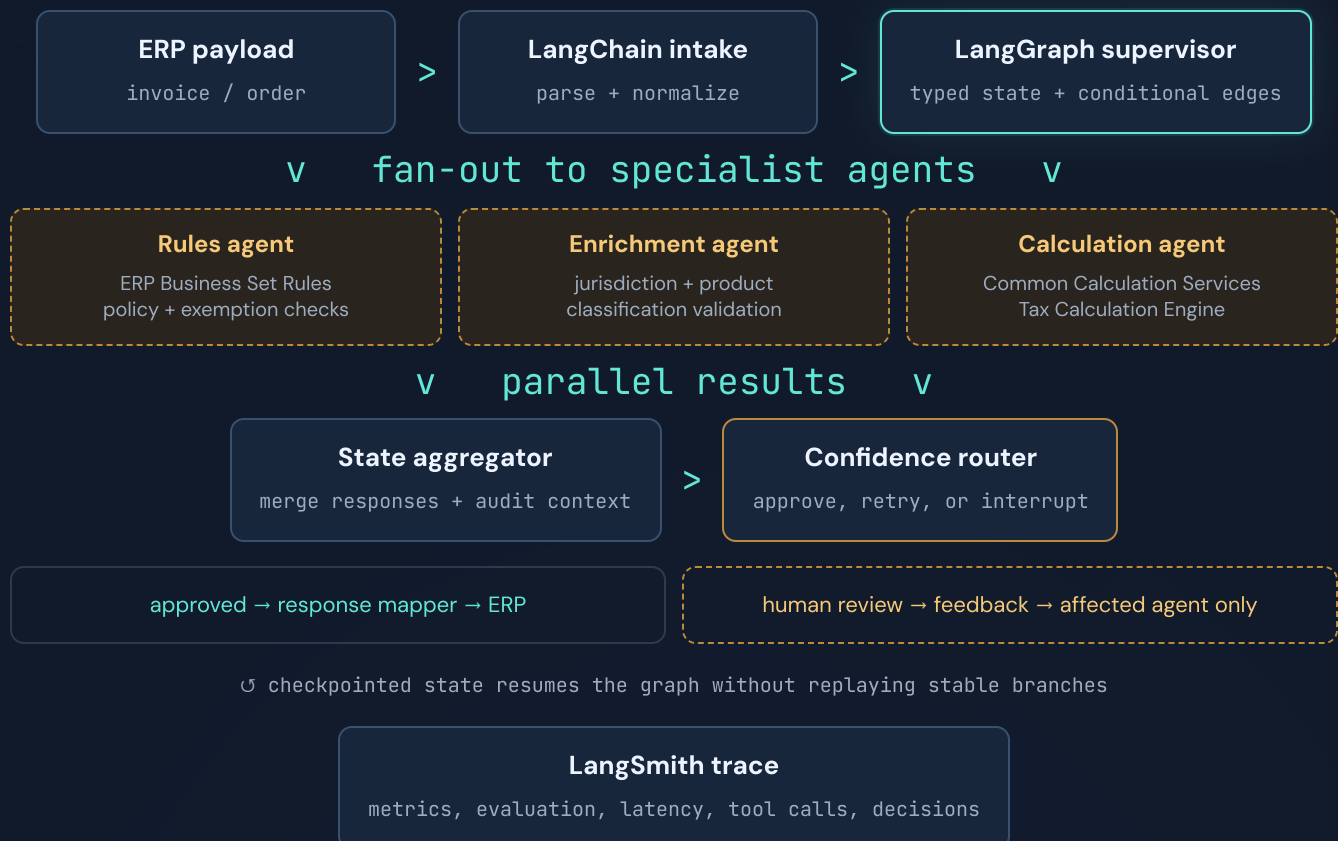
| TOOL CALL BREAKDOWN  |       |
|----------------------|-------|
| get_hotel_reviews    | 3     |
| search_hotels        | 2     |
| search_activities    | 2     |
| get_weather_forecast | 2     |
| calculate_budget     | 2     |
| convert_currency     | 1     |
| TOKEN USAGE          |       |
| Prompt (input)       | 5,812 |
| Completion (output)  | 8,286 |

**The 2 "interrupted" runs?** Those are the human\_review graph nodes—interrupt\_before firing exactly as designed. LangSmith traces the GraphInterrupt event with the full serialized travel plan payload, so you can inspect every approval gate in the trace timeline. Not errors—the **stateful human-in-the-loop architecture** working as intended.

#### // APPLYING THE PATTERN TO TAX CALCULATION

The same architecture can orchestrate a real **"tax calculation workflow"**. Instead of treating tax as one opaque API call, the graph can coordinate the full journey from an ERP transaction payload to a validated tax response that is ready to return to the ERP.

#### TAX CALCULATION – LANG FAMILY ORCHESTRATION FLOW



#### GOVERNED AI HARNESS AND TAX SERVICE TOOLS

ERP Business Set Rules   Common Calculation Services   Tax Calculation Engine   Platform Services

Validation & Retry Tools   Human Feedback

- > **ERP intake and payload parsing** — A LangChain tool validates the inbound invoice or sales-order payload, normalizes customer, ship-from, ship-to, product, exemption, jurisdiction, currency, and line-item data, and records the original request for traceability.
- > **Tax connector orchestration** — LangGraph routes the normalized transaction through the required connector layers, preserving correlation IDs, connector responses, retries, and error state as explicit graph state rather than hidden integration code.
- > **Business and tax calculation services** — Specialized tools can invoke ERP Business Set Rules, the Tax Calculation Engine, Common Calculation Services, and supporting platform services in the correct dependency order.
- > **Exception and confidence routing** — The supervisor can detect missing jurisdiction data, ambiguous product taxability, exemption conflicts, calculation failures, or policy exceptions and route only the affected branch for enrichment or retry.
- > **Human approval and feedback** — For high-value transactions, low-confidence classifications, or policy exceptions, LangGraph pauses at a durable human approval gate. The reviewer can approve, correct, or provide feedback that re-routes the affected tax branch without replaying the entire workflow.
- > **ERP-ready output** — A response-mapping tool assembles the tax result into the ERP contract, including tax jurisdiction, taxable basis, rates, calculated tax, exemptions, warnings, audit metadata, and correlation details.

**The AI harness around the workflow:** LangChain supplies the structured tools, parsers, validators, connector adapters, and policy prompts. LangGraph provides the typed state, conditional routing, parallel service calls, retries, checkpoints, human approval, and feedback loop. LangSmith provides real-time traces, latency and failure metrics, token and tool usage, evaluation datasets, and end-to-end visibility from ERP intake to ERP response. The AI remains inside governed tools and approval boundaries; it orchestrates the workflow without replacing authoritative tax calculation services.

If you're building multi-agent systems and haven't explored LangGraph's **state-driven orchestration model**, I'd strongly recommend it. The **conditional edge routing**, sub-graph composition, graph-native checkpointing, and **real-time LangSmith observability** solve orchestration problems that most agentic frameworks leave entirely to the developer.

The full architecture walkthrough—graph topology, agent design, tool integration patterns, and LangSmith evaluation setup—is on my GitHub. Happy to discuss the architectural trade-offs.

What's the most complex **multi-agent orchestration challenge** you've tackled? I'd love to hear about your graph architectures, supervisor patterns, and observability setups in the comments.

#AgenticAI #MultiAgentArchitecture #SupervisorPattern #StatefulGraphOrchestration #GraphNativeAI  
#StateGraph #LangChain #LangGraph #LangSmith #ReActPattern #HumanInTheLoop #SelectiveReExecution  
#ConditionalEdgeRouting #SubGraphComposition #AIOrchestration #RealTimeObservability #AgentEvaluation  
#Claude #Anthropic #Tavily #DuckDuckGo #MemorySaver #Gradio #AIEngineering #LLMApplications  
#BuildInPublic



Like



Comment



Repost



Send

[Copy plain text for LinkedIn](#)